Software Engineering Lab-5

Static Code Analysis - School Bus Transport Register

PES2UG24CS461

Test Reports:

Before:

SE_Lab_PES2UG24CS461 > Lab5 > reports > before > bandit_report.txt

```
Run started:2026-10-14 06:31:48.804616+00:00

Test results:
>> Issue: [B404:blacklist] Consider possible security implications associated with the subprocess module.
   Severity: Low   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess
   Location: .\school_transport.py:1:0
1  import subprocess
2  import sqlite3
3

--------------------------------------------------
>> Issue: [B608:h                    ;ions] Possible SQL injection vector through string-based query construction.
   Severity: Medi  Chat Ctrl+L  ···  lium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\school_transport.py:9:19
8  def enrollRider(student,route,stop,fee):
9      conn.execute(f"INSERT INTO riders (student, route, stop, fee) VALUES ('{student}', '{route}', '{stop}', {fee})")
10     conn.commit()

--------------------------------------------------
>> Issue: [B602:subprocess_popen_with_shell_equals_true] subprocess call with shell=True identified, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b602_subprocess_popen_with_shell_equals_true.html
   Location: .\school_transport.py:24:4
23 def dispatchAlert(route):
24     subprocess.run("echo dispatching bus " + route, shell=True, check=False)
25

--------------------------------------------------
>> Issue: [B101:assert_used] Use of assert detected. The enclosed code will be removed when compiling to optimised byte code.
   Severity: Low   Confidence: High
   CWE: CWE-703 (https://cwe.mitre.org/data/definitions/703.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b101_assert_used.html
   Location: .\school_transport.py:38:4
37     enrollRider("Zoya", "R2", "Fort Gate", 2100)
38     assert feeCollected("R1") == 3600
39     exportRoster("route_r1.txt", "R1")

--------------------------------------------------

Code scanned:
   Total lines of code: 30
   Total lines skipped (#nosec): 0
   Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
   Total issues (by severity):
      Undefined: 0
      Low: 2
      Medium: 1
      High: 1
   Total issues (by confidence):
      Undefined: 0
      Low: 0
      Medium: 1
      High: 3
Files skipped (0):
```

SE_Lab_PES2UG24CS461 > Lab5 > reports > before > flake8_report.txt

```
school_transport.py:5:80: E501 line too long (110 > 79 characters)
school_transport.py:8:24: E231 missing whitespace after ','
school_transport.py:8:30: E231 missing whitespace after ','
school_transport.py:8:35: E231 missing whitespace after ','
school_transport.py:9:80: E501 line too long (116 > 79 characters)
school_transport.py:14:80: E501 line too long (95 > 79 characters)
school_transport.py:23:1: E302 expected 2 blank lines, found 1
school_transport.py:29:80: E501 line too long (98 > 79 characters)
school_transport.py:30:5: E722 do not use bare 'except'
```

SE_Lab_PES2UG24CS461 > Lab5 > reports > before > pylint_report.txt

```
************* Module school_transport
school_transport.py:5:0: C0301: Line too long (110/100) (line-too-long)
school_transport.py:9:0: C0301: Line too long (116/100) (line-too-long)
school_transport.py:1:0: C0114: Missing module docstring (missing-module-docstring)
school_transport.py:8:0: C0116: Missing function or method docstring (missing-function-docstring)
school_transport.py:8:0: C0103: Function name "enrollRider" doesn't conform to snake_case naming style (invalid-name)
school_transport.py:13:0: C0116: Missing function or method docstring (missing-function-docstring)
school_transport.py:13:0: C0103: Function name "routeRoster" doesn't conform to snake_case naming style (invalid-name)
school_transport.py:13:23: W0613: Unused argument 'driver' (unused-argument)
school_transport.py:17:0: C0116: Missing function or method docstring (missing-function-docstring)
school_transport.py:17:0: C0103: Function name "exportRoster" doesn't conform to snake_case naming style (invalid-name)
school_transport.py:18:10: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
school_transport.py:18:10: R1732: Consider using 'with' for resource-allocating operations (consider-using-with)
school_transport.py:23:0: C0116: Missing function or method docstring (missing-function-docstring)
school_transport.py:23:0: C0103: Function name "dispatchAlert" doesn't conform to snake_case naming style (invalid-name)
school_transport.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
school_transport.py:27:0: C0103: Function name "feeCollected" doesn't conform to snake_case naming style (invalid-name)
school_transport.py:30:4: W0702: No exception type(s) specified (bare-except)
school_transport.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)

-------------------------------------------------------------------
Your code has been rated at 4.00/10 (previous run: 4.00/10, +0.00)
```

After:

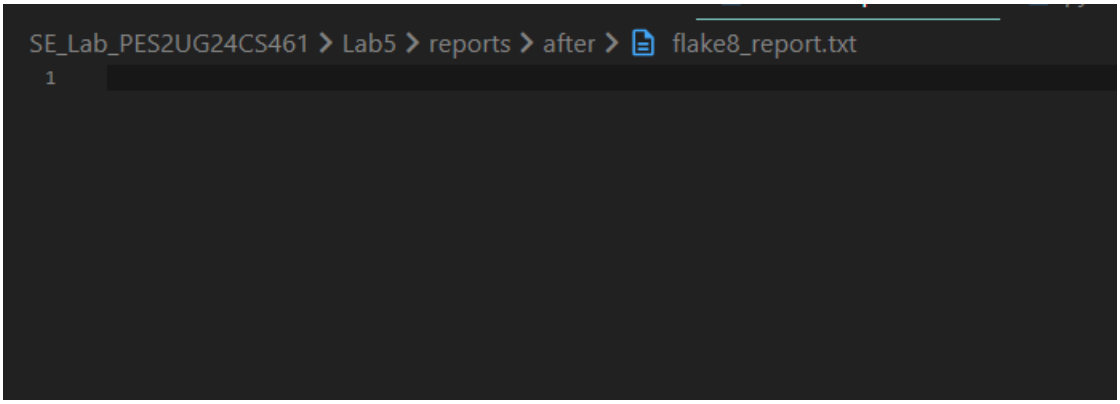
SE_Lab_PES2UG24CS461 > Lab5 > reports > after > bandit_report.txt

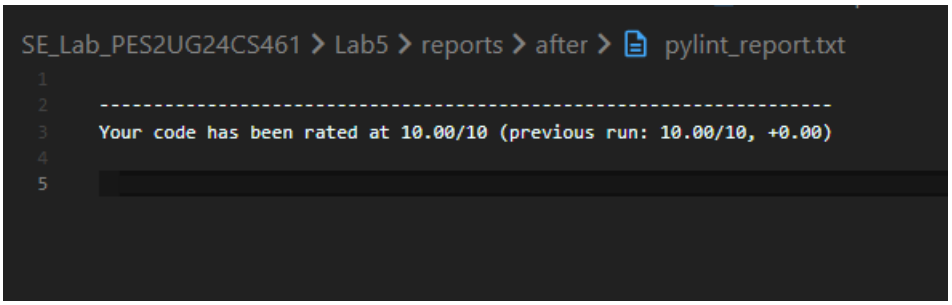
```
Run started:2026-10-14 09:51:34.182196+00:00

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 59
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```





Issues Log:

Lab 5 Issues Log: School Bus Transport Register

Issue	Type	Line(s)	Description	Fix Approach
Insecure Command Execution (`shell=True`)	Security (Bandit B602)	24	`subprocess.run` is invoked with `shell=True` and string concatenation (`"echo dispatching bus " + route`), exposing the program to command injection.	Avoid invoking a shell; replace with a safe list of command arguments or standard Python print/logging without shell execution.
SQL Injection Risk	Security (Bandit B608)	9	`conn.execute` uses an f-string to concatenate user inputs directly into an SQL statement (`INSERT INTO riders ... VALUES ('{student}', ...)`).	Use parameterized SQL queries (`?` placeholders) and pass variables as a tuple to ensure safe parameter binding.
Subprocess Module Import & Assert Usage	Security / Quality (Bandit B404 / B101)	1, 38	Importing subprocess when unnecessary creates potential attack surface; `assert` statements are stripped when Python bytecode is optimized (`python -O`).	Remove subprocess if dispatch logic can be handled natively, or safely handle process invocation without shell; replace verification assertion with explicit conditional check / `ValueError`.
Bare Except Clause	Bug / Code Quality (Pylint W0702, Flake8 E722)	30	`except:` catches all exceptions indiscriminately including system-exiting exceptions (`KeyboardInterrupt`, `SystemExit`) and masks underlying syntax/database errors.	Catch specific exceptions such as `sqlite3.Error` or `sqlite3.OperationalError` instead of using a bare `except:`.
Unclosed File Resource & Encoding Omission	Resource Leak / Quality (Pylint W1514, R1732)	18	`open()` is called without a context manager (`with open(...)`) and without specifying explicit encoding (`encoding="utf-8"`), risking file handle leaks and cross-platform encoding bugs.	Use `with open(path, "w", encoding="utf-8") as out:` to guarantee file closure and reproducible encoding.
Unused Parameter	Quality (Pylint W0613)	13	`driver` parameter in `routeRoster(route, driver)` is never used within the function body.	Remove the unused `driver` parameter or utilize it if required, updating all caller signatures accordingly.
PEP 8 Formatting & Naming Style Violations	Style (Flake8 E501, E231, E302; Pylint C0103, C0301)	5, 8, 9, 14, 17, 23, 27, 29	Long lines exceeding 79/100 chars, camelCase function names (`enrollRider`, `routeRoster`, `exportRoster`, `dispatchAlert`, `feeCollected`) instead of PEP 8 `snake_case`, missing spaces after commas, and improper blank lines between functions.	Rename functions to `snake_case` (or provide aliases if needed), wrap long SQL statements onto multiple lines, ensure 2 blank lines between top-level functions, and fix comma spacing.
Missing Docstrings	Style / Documentation (Pylint C0114, C0116)	1, 8, 13, 17, 23, 27, 34	Module and functions lack docstrings explaining their purpose, arguments, and return values.	Add descriptive docstrings to the module and every function.

Reflection Question:

Q1: What are the key differences between the findings reported by Pylint, Flake8, and Bandit?

Each tool checks a different part of the code. Flake8 checks for style, formatting, and PEP 8 rules like line length, spacing after commas, and blank lines. Pylint is a deeper code quality checker that looks at code design, unused variables, missing docstrings, and bad practices like not using 'with' when opening files. Bandit is purely a security scanner that finds vulnerabilities like SQL injection, unsafe shell commands, and insecure imports.

Q2: Why are vulnerabilities like SQL injection and Command Injection particularly critical in real-world systems?

Both vulnerabilities let untrusted user inputs run as code. With SQL injection, an attacker can steal private student records, change fees, or delete whole database tables. With Command injection (using shell=True), an attacker can run any command they want on the server's operating system, which can let them take full control of the machine.

Q3: How does adopting static analysis early in the Software Development Life Cycle (SDLC) benefit software teams?

Fixing bugs early while writing code is much easier and cheaper than fixing them after the app goes live. It also keeps code clean and readable across the entire team, and avoids spending time in code reviews arguing about simple formatting rules.

Q4: What challenges might a development team encounter when introducing static analysis to a legacy codebase, and how can they be mitigated?

If you run these tools on old code, they can show hundreds or thousands of errors at once, which can overwhelm developers. Also, some warnings might be false alarms. A good way to handle this is to only check new and modified files first, and slowly clean up older parts. Teams can also configure the rules file to turn off rules that are not relevant to their project.